

Reconfigurable Computing — Post Mid-Semester Lecture Notes

(Lectures 19–26)

Expanded with external literature

Based on course slides by Dr. Meetha V. Shenoy
Added references: routing, PUFs, hardware Trojan, CGRA

Contents

1	Lecture 19: FPGA Routing and Architecture	2
1.1	Overview of FPGA CAD Flow	2
1.2	Island-Style FPGA Architecture	2
1.3	Switchbox Topologies and Trade-offs	2
1.4	Routing as a Graph Problem	2
1.5	Pathfinder: Negotiation-Based Router	2
1.6	Maze / BFS / Dijkstra Variant Routers	3
1.7	Recent Research Directions	3
2	Lecture 20: ZedBoard, Zynq SoC and JTAG	4
2.1	ZedBoard and Zynq Recap	4
2.2	Boundary-Scan (IEEE 1149.1) and JTAG	4
2.3	Secure Considerations for JTAG	4
3	Lecture 21–22: FPGA Security (Expanded)	5
3.1	Supply-Chain Threats and Attack Vectors	5
3.2	Hardware Trojan Taxonomy and Detection	5
3.3	PUFs (Physically Unclonable Functions)	5
3.4	Secure Boot and Bitstream Authentication	6
3.5	Design-time Defenses	6
4	Lecture 23: Coarse-Grained Reconfigurable Architectures (CGRA) — Expanded	7
4.1	CGRA Recap	7
4.2	Architectural Design Choices and Trade-offs	7
4.3	Use-cases and Examples	7
5	Lectures 24–26: Configurable Logic Block (CLB) Architecture — Expanded	8
5.1	CLB and Slice Recap	8
5.2	LUT-as-RAM and SRLs	8
5.3	Block RAM (BRAM) vs Distributed RAM Trade-offs	8
5.4	Carry Chains and Fast Arithmetic	8
5.5	Pipelining and Throughput-Optimized Design	8

6	Lecture 26 Addendum: Implementing 3×3 Convolution on FPGA (Expanded)	9
6.1	Problem and Resource Mapping	9
6.2	Throughput and Latency	9
6.3	Quantization and Fixed-Point Design	9
7	References	10

1 Lecture 19: FPGA Routing and Architecture

1.1 Overview of FPGA CAD Flow

According to the flow diagram shown in the slides (Design Entry → Synthesis → Logic Optimization → Mapping to k-LUT → Packing LUTs to CLBs → Placement → Routing → Simulation → Configuration) the FPGA compilation process proceeds through the usual EDA steps (synthesis, tech-mapping, placement, routing). The course slides provide a compact workflow diagram and discussion of placement & routing stages. :contentReference[oaicite:5]index=5

1.2 Island-Style FPGA Architecture

Island-style FPGAs are organized as a 2-D array of logic blocks (CLBs) with horizontal and vertical routing channels. Connection boxes link CLB pins to channel wires and switch boxes connect channel wires. This is illustrated in the course slides (island-style figure). :contentReference[oaicite:6]index=6

1.3 Switchbox Topologies and Trade-offs

Switch box design is a major architecture decision. Common topologies:

- **Subset / Planar switch boxes:** connect track i to track i only (cheap, area-efficient).
- **Wilton switch boxes:** a permuted mapping that improves routability for certain designs.
- **Universal switch boxes:** full connectivity between domains (best routability, large area).

Studies comparing these topologies show Wilton often achieves a good routability/area trade-off, while universal boxes provide highest routability at area cost. See formal evaluations and layout analyses for details. :contentReference[oaicite:7]index=7

Design parameter: Connection flexibility (F_c)

The connection flexibility F_c measures how many track wires a CLB pin can connect to. Increasing F_c improves routing success probability for dense nets but raises area and bitstream complexity. The course slides highlight this parameter in the context of connection boxes. :contentReference[oaicite:8]index=8

1.4 Routing as a Graph Problem

Routing is modeled as a graph; nodes represent pins, wire segments, and switches. Routing must avoid obstacles (occupied resources) and optimize cost metrics (wirelength, delay, congestion). Practical routers use multi-stage approaches (global routing followed by detailed routing). The slides give a pedagogical description of these phases. :contentReference[oaicite:9]index=9

1.5 Pathfinder: Negotiation-Based Router

One of the canonical FPGA routers is the **Pathfinder** algorithm: it performs initial shortest-path routing and then iteratively negotiates congestion by raising the cost of overused resources and rerouting nets. This negotiation-based approach balances routability and performance; further studies have refined and revisited details of PathFinder to reduce delay variability and improve

convergence. The original PathFinder paper is the foundational description, and later analysis (e.g., quantifying delay noise and tuning ‘pressure’ factors) provides practical guidance for tool tuning. [:contentReference\[oaicite:10\]index=10](#)

Practical implications for FPGA design

- Long, wide buses and high-fanout nets often require manual floorplanning or constraints to avoid catastrophic routing congestion.
- Synthesis choices (logic duplication, buffer insertion) affect routability and timing closure.

1.6 Maze / BFS / Dijkstra Variant Routers

Detail-level routing often uses BFS-like expansion (maze routing) with cost heuristics (distance, congestion). These methods are used inside PathFinder iterations or as parts of other router designs. The tutorial on FPGA routing provides a clear, actionable description of domain negotiation and DFS/BFS strategies to find legal routes in subset switch boxes. [:contentReference\[oaicite:11\]index=11](#)

1.7 Recent Research Directions

Current research looks at:

- SAT-based and ILP approaches to embed complex nets and accelerate routing quality for specific fabrics. [:contentReference\[oaicite:12\]index=12](#)
- Improved switch box designs (turn-restricted, grouped routing), which trade area, delay, and ease-of-routing. [:contentReference\[oaicite:13\]index=13](#)

2 Lecture 20: ZedBoard, Zynq SoC and JTAG

2.1 ZedBoard and Zynq Recap

ZedBoard pairs a PS (dual-core ARM Cortex-A9) with PL (Artix-7 fabric). PS–PL communication is via AXI interconnects; typical flows use ARM to offload compute kernels to PL using AXI-Stream/MM interfaces. See course slides for labeled connectors and board programming options (USB-JTAG, SPI flash, SD). :contentReference[oaicite:14]index=14

2.2 Boundary-Scan (IEEE 1149.1) and JTAG

Boundary-scan cells create a serial chain around I/O pins and allow several test modes: EXTEST, INTEST, BYPASS. TAP signals (TCK, TMS, TDI, TDO, optional TRST) control the finite-state TAP controller. Slides illustrate wrapper architecture and common test modes. :contentReference[oaicite:15]index=15

2.3 Secure Considerations for JTAG

Because JTAG provides low-level access, production systems often disable or protect JTAG (via fuses or secure configuration) to prevent extraction of sensitive bitstreams or runtime introspection.

3 Lecture 21–22: FPGA Security (Expanded)

3.1 Supply-Chain Threats and Attack Vectors

FPGA-based systems are vulnerable across the supply chain: foundries, IP vendors, EDA tools, integrators, and field updates all introduce trust boundaries. The slides describe cloning, replay attacks, and backdoors. We add structured threat classes:

- **Bitstream theft / cloning:** attackers copy the bitstream from flash or during programming flows.
- **Hardware Trojans:** malicious modifications inserted at RTL, netlist, or GDSII level.
- **Backdoors / Malicious IP:** vendor IP with hidden interfaces or covert channels.
- **Replay / rollback attacks:** reprogramming older bitstreams to re-enable known vulnerabilities.
- **Reverse engineering / side-channels:** extracting secrets via power, EM, or physical attacks.

3.2 Hardware Trojan Taxonomy and Detection

A canonical taxonomy partitions Trojans by:

- **Insertion stage:** design-time, integration, or fabrication.
- **Trigger:** input pattern, time-based, sensor-based, rare event.
- **Payload:** leak, disable, degrade, or change functionality.

Detection methods include structural analysis, logic testing, side-channel analysis (power/EM), formal methods, and runtime monitoring. Standard surveys and reviews compile and evaluate these techniques—useful references summarize detection capabilities and limitations. :contentReference[oaicite:17]index=17

3.3 PUFs (Physically Unclonable Functions)

PUFs exploit manufacturing variability to produce a device-unique response to challenges (challenge-response pairs, CRPs). Two common PUF families:

- **Ring-oscillator PUFs (RO-PUF):** compare frequencies of oscillator pairs to produce bits.
- **Arbiter PUFs:** race paths through multiplexers producing a stable 0/1 depending on path delay differences.

Important performance metrics:

- **Uniqueness:** inter-device Hamming distance (ideal $\approx 50\%$).
- **Reliability / stability:** intra-device consistency across temperature/voltage.
- **Entropy / CRP space:** scalability of secure challenge–response pairs.

Reviews provide experimental data and robustness techniques (helper data, fuzzy extractors) to convert noisy PUF outputs into stable keys. RO-PUFs are practical in FPGAs but often require calibration/conditioning for robust operation. :contentReference[oaicite:18]index=18

3.4 Secure Boot and Bitstream Authentication

Best practices:

- Use signature verification during boot (image signed by vendor key; device verifies using public key in ROM/fuses).
- Encrypt bitstreams in storage and use on-chip key material (PUFs or secure fuses) for decryption.
- Include monotonic counters or versioning to prevent rollback/replay. :contentReference[oaicite:19]index=19

3.5 Design-time Defenses

- IP vetting and provenance: prefer vetted open-source IP or signed vendor IP.
- Formal verification and equivalence checking between RTL and netlist.
- Runtime monitors (watchdogs and integrity checks).

Surveys recommend combining multiple detection/mitigation layers for stronger coverage. :contentReference[oaicite:20]index=20

4 Lecture 23: Coarse-Grained Reconfigurable Architectures (CGRA) — Expanded

4.1 CGRA Recap

CGRA is a mid-point between CPUs and FPGAs: arrays of PEs with reconfigurable interconnect provide operation-level reconfiguration (ALU/MAC units as building blocks). They accelerate kernels with regular dataflow, while reducing compilation complexity compared to bit-level FPGA design. Course slides summarize PE arrays, context memories, and host controllers. :contentReference[oaicite:21]index=21

4.2 Architectural Design Choices and Trade-offs

Key design axes for CGRAs:

- **PE complexity:** simple ALUs vs richer arithmetic units.
- **Interconnect topology:** nearest-neighbor (cheap, local), crossbars (flexible, expensive), or hybrid augmented networks.
- **Context memory and reconfiguration granularity:** per-PE contexts vs block-level contexts.
- **Compiler support:** mapping high-level loops to the PE array efficiently is essential.

Surveys highlight that CGRA efficiency depends strongly on compiler technology and data-movement optimizations. :contentReference[oaicite:22]index=22

4.3 Use-cases and Examples

- DSP pipelines (FIR, convolution).
- CNN inference accelerators (when integer/low-precision ops dominate).
- Stream processing where dataflow is statically known.

5 Lectures 24–26: Configurable Logic Block (CLB) Architecture — Expanded

5.1 CLB and Slice Recap

Modern 7-series FPGA slices contain:

- Four LUTs (LUT6 primitives or pairable LUT5s).
- Eight flip-flops.
- Carry logic (fast chains).
- Wide multiplexers and optional LUTRAM/SRL in SLICEM. Slides show diagrams of SLICEM vs SLICEL internals.

5.2 LUT-as-RAM and SRLs

- **Distributed RAM (LUTRAM):** small, low-latency memories implemented in SLICEM LUTs.
- **SRL32 / SRLC32E:** LUT configured as a 32-bit deep shift register with an addressable tap. Cascading yields deeper delays (up to 128 cycles per SLICEM with four LUTs). These primitives are invaluable for FIFOs, line buffers, and small delays without consuming BRAM.

5.3 Block RAM (BRAM) vs Distributed RAM Trade-offs

- BRAM: large capacity, synchronous read/write, good for frame buffers and large FIFOs.
- Distributed RAM (LUTRAM): small capacity, asynchronous reads possible, located near logic — good for small tables, coefficients, and low-latency accesses.

Tables in the slides summarize relative capacities and use-cases; designers choose based on width/depth and latency needs. :contentReference[oaicite:25]index=25

5.4 Carry Chains and Fast Arithmetic

Dedicated carry logic (CARRY4 blocks) enables very fast adders and accumulation trees; use carry-aware synthesis to leverage these primitives for area and timing improvements. Slides show how LUT outputs feed carry logic for optimized adders. :contentReference[oaicite:26]index=26

5.5 Pipelining and Throughput-Optimized Design

Design guidelines:

- Pipeline across DSPs, LUT logic, and BRAM interfaces to meet target clock.
- Balance pipeline stages to minimize latency and maximize throughput.
- Use HLS pragmas or RTL pipeline insertion for loop pipelining and II reduction.

Slides include an example pipeline for a multiply-add expression and discuss initiation interval and latency concepts. :contentReference[oaicite:27]index=27

6 Lecture 26 Addendum: Implementing 3×3 Convolution on FPGA (Expanded)

6.1 Problem and Resource Mapping

For a 28×28 grayscale image and 3×3 kernel:

- Use BRAM for line buffers (store rows).
- Use SRL32 for sliding-window shift registers per row to stream pixels cheaply.
- Use DSP48E1 for 9 parallel multiplications per output; if DSP count limited, time-multiplex DSPs.
- Use carry-chain adders for partial and final accumulation.

The slide set presents a stage-by-stage pipeline; the extended notes emphasize memory-bank schemes and resource-time trade-offs. :contentReference[oaicite:28]index=28

6.2 Throughput and Latency

- With full spatial parallelism (9 multipliers per cycle), throughput = 1 output pixel / cycle (after pipeline fill).
- If multipliers are time-multiplexed, throughput reduces proportionally.
- Memory bandwidth (to/from BRAM or DRAM) can become the bottleneck; use burst transfers and wide ports to maximize utilization.

6.3 Quantization and Fixed-Point Design

When mapping to FPGA, quantize coefficients and intermediate results to fixed-point representation. Choose bit-widths after range analysis and simulate for accuracy vs resource trade-off (DSP width and BRAM depth). Use shift-and-add or DSP pre-adder features for efficient multiplies. :contentReference[oaicite:29]index=29

7 References

Slides (uploaded course material):

- Lecture 19 slides (FPGA routing, architecture). :contentReference[oaicite:30]index=30
- Lecture 20 slides (ZedBoard & JTAG). :contentReference[oaicite:31]index=31
- Lecture 21 slides (Security Part 1 — PUF, Trojans). :contentReference[oaicite:32]index=32
- Lecture 22 slides (Security Part 2 — obfuscation, tamper). :contentReference[oaicite:33]index=33
- Lecture 23 slides (CGRA). :contentReference[oaicite:34]index=34
- Lecture 24 slides (CLB Part 1). :contentReference[oaicite:35]index=35
- Lecture 25 slides (CLB Part 2). :contentReference[oaicite:36]index=36
- Lecture 26 slides (CLB Part 3 & convolution example). :contentReference[oaicite:37]index=37

Selected external literature referenced in these expanded notes:

- PathFinder: W. Kong and others, *PathFinder: a negotiation-based performance-driven router*, ACM/IEEE (canonical reference). :contentReference[oaicite:38]index=38
- FPGA routing tutorial and switch-box analysis (survey/tutorial). :contentReference[oaicite:39]index=39
- Quantifying delay noise and tuning PathFinder (analysis of VPR/PathFinder). :contentReference[oaicite:40]index=40
- PUF review: review paper covering RO-, SRAM-, arbiter-PUFs, and metrics. :contentReference[oaicite:41]index=41
- RO-PUF recent robustness work and variants. :contentReference[oaicite:42]index=42
- Hardware Trojan taxonomy and detection surveys (Tehranipoor & Koushanfar and related surveys). :contentReference[oaicite:43]index=43
- CGRA surveys and architecture overviews (ADRES and other templates). :contentReference[oaicite:44]index=44
- Switch-box layout and SAT/Switch comparisons (subset, Wilton, universal). :contentReference[oaicite:45]index=45